In a Gradle or Maven project, test classes are usually placed in a src/test/java folder. In Spring, unit tests aren't required to load the Spring application context, and they don't rely on any Spring library. On the other hand, integration tests need a Spring application context to run. This section will show you how to test a RESTful application like Catalog Service with unit and integration tests.

3.4.1 Unit tests with JUnit 5

Unit tests are not aware of Spring and don't rely on any Spring library. They are intended to test the behavior of single components as isolated units. Any dependency at the edge of the unit is mocked to keep the test shielded from external components.

Writing unit tests for Spring applications is no different from writing them for any other Java application, so I won't go into detail about them. By default, any Spring project created from Spring Initializr contains the `spring-boot-starter-test` dependency, which imports testing libraries like JUnit 5, Mockito, and AssertJ into the project. So we're all set for writing unit tests.

The business logic of an application is usually a sensible area to cover with unit tests. In the Catalog Service application, a good candidate for unit testing might be the validation logic for the `Book` class. The validation constraints are defined using the Java Validation API annotations, and we are interested in testing that they are applied correctly to the `Book` class. We can check that in a new `BookValidationTests` class, as shown in the following listing.

Listing 3.15 Unit tests to verify book validation constraints